

Proper Notes Format

◆ STEP 8 : CONTROLLER CALLS THE SERVICE LAYER

Now the controller has received the email text.

Inside your method:

```
return ResponseEntity.ok(
    emailService.analyze(
        request.getEmailText()
    )
);
```

Let's focus on:

```
emailService.analyze(
    request.getEmailText()
);
```

What is Happening Here?

The controller is saying:

"I have received the request. Now let the service do the actual work."

Remember:

Controller

handles requests.

Service

contains business logic.

Why Not Put Everything Inside Controller?

You could write:

```
@PostMapping("/analyze")
public ResponseEntity<EmailResponseDTO> analyze(...) {
    // Call Flask
    // Process response
    // Build DTO
    // Return result
}
```

But this becomes messy.

Instead:

```
Controller
↓
Service
```

Controller handles requests.

Service handles business logic.

Restaurant Example

```
Customer
↓
Waiter
↓
Chef
```

Customer:

```
User
```

Waiter:

```
EmailController
```

Chef:

```
EmailService
```

The **waiter** does not cook.

The **chef** cooks.

Similarly:

Controller does not analyze emails.

Service analyzes emails.

What Value Is Passed?

```
request.getEmailText()
```

returns:

```
"You won a lottery"
```

So Spring executes:

```
emailService.analyze(
    "You won a lottery"
);
```

Control Moves To EmailService

Execution jumps from:

```
EmailController
```

to:

```
EmailService
```

and starts running:

```
analyze(...)
```

inside EmailService.

◆ STEP 9 : EMAILSERVICE CALLS FLASK

Inside EmailService you have something similar to:

```
restTemplate.postForEntity(...)
```

What is RestTemplate?

RestTemplate is a **Spring class used to call external APIs.**

Think of it as a messenger.

Example

Suppose your Spring application wants to talk to Flask.

```
Spring
↓
Flask
```

It cannot simply call:

```
predict()
```

because Flask is a completely separate application.

Instead Spring sends an HTTP request.

Just like Swagger sends requests to Spring.

Communication Flow

```
Swagger
↓
Spring
↓
Flask
```

Request Sent To Flask

Spring sends:

```
POST http://localhost:5000/predict
```

with:

```
{
  "emailText": "You won a lottery"
}
```

Why localhost:5000?

Usually:

```
Spring Boot
```

runs on:

```
localhost:8080
```

and:

```
Flask
```

runs on:

```
localhost:5000
```

Different applications.

Different ports.

What Happens Next?

Spring waits for Flask's response.

Flow

```
Spring
  ↓
POST /predict
  ↓
Flask
```

◆ STEP 10 : FLASK RECEIVES THE REQUEST

Flask receives:

```
{
  "emailText": "You won a lottery"
}
```

Flask endpoint:

/predict

starts executing.

What Does Flask Do?

Flask extracts:

You won a lottery

from the request.

Then Flask sends it to the Machine Learning model.

Flow

```
Request
↓
Flask API
↓
ML Model
```

◆ STEP 11 : MACHINE LEARNING MODEL MAKES PREDICTION

The ML model analyzes the email.

Example:

"You won a lottery"

contains words often found in spam emails.

Model predicts:

SPAM

Another example:

Meeting scheduled for tomorrow

might return:

HAM

What Is HAM?

In spam filtering:

SPAM

means unwanted email.

HAM

means legitimate email.

Result Produced

Model generates:

SPAM

◆ STEP 12 : FLASK RETURNS RESPONSE

Flask now sends a response back to Spring.

Example:

```
{
  "prediction": "SPAM"
}
```

Response Flow

```
ML Model
  ↓
Flask
  ↓
Spring
```

Spring receives:

```
{
  "prediction": "SPAM"
}
```

Important Point

At this moment:

Spring has successfully received the ML prediction.

Now it must convert that prediction into your project's response format.

◆ STEP 13 : EMAILSERVICE CREATES EmailResponseDTO

Inside EmailService:

Prediction:

SPAM

Service creates:

```
new EmailResponseDTO(
    "SPAM",
    1.0
);
```

What Is EmailResponseDTO?

A DTO used for sending response data.

Object:

```
EmailResponseDTO
```

contains:

```
result = SPAM
confidence = 1.0
```

Visual Representation

```
EmailResponseDTO
├── result = SPAM
└── confidence = 1.0
```

Service Returns Object

```
return new EmailResponseDTO(
    "SPAM",
    1.0
);
```

Control goes back to:

```
EmailController
```

◆ STEP 14 : CONTROLLER RECEIVES RESULT

Controller originally called:

```
emailService.analyze(...)
```

Now EmailService returns:

```
new EmailResponseDTO(
    "SPAM",
    1.0
);
```

So this line:

```
emailService.analyze(
    request.getEmailText()
)
```

produces:

```
EmailResponseDTO
```

object.

Then Controller Executes

```
ResponseEntity.ok(...)
```

Which becomes:

```
ResponseEntity.ok(
    new EmailResponseDTO(
        "SPAM",
        1.0
    )
);
```

◆ STEP 15 : WHAT DOES ResponseEntity.ok() DO?

```
ResponseEntity.ok(...)
```

means:

"Everything was successful."

Spring automatically sets:

```
HTTP Status = 200 OK
```

Why Use **ResponseEntity**?

Without it:

```
return dto;
```

Only data is returned.

With:

```
ResponseEntity.ok(dto);
```

Spring returns:

```
Status Code
+
Response Body
```

Result

Status:

200 OK

Body:

```
{
  "result": "SPAM",
  "confidence": 1.0
}
```

◆ STEP 16 : SPRING CONVERTS OBJECT INTO JSON

Controller returns:

EmailResponseDTO

object.

But browsers and APIs understand:

JSON

not Java objects.

Spring automatically converts:

EmailResponseDTO

into:

```
{
  "result": "SPAM",
  "confidence": 1.0
}
```

This conversion is called:

Serialization

Why Does It Happen Automatically?

Because of:

@RestController

Spring knows:

"Convert returned objects into JSON."

◆ STEP 17 : RESPONSE RETURNS TO SWAGGER

Final flow:

```

Controller
↓
Tomcat
↓
Swagger

```

Swagger receives:

```
Status: 200 OK
```

and

```

{
  "result": "SPAM",
  "confidence": 1.0
}

```

User Sees

```

{
  "result": "SPAM",
  "confidence": 1.0
}

```

on the screen.

◆ COMPLETE FLOW

```

Swagger
↓
POST /email/analyze
↓
Tomcat
↓
Spring Routing Table
↓
EmailController.analyze()
↓
@RequestBody
↓
EmailRequestDTO
↓
EmailService.analyze()
↓
RestTemplate
↓
Flask API
↓
ML Model
↓
SPAM
↓
EmailResponseDTO
↓
ResponseEntity.ok()
↓
JSON Response
↓
Swagger

```

Key Idea

```
@RequestMapping("/email")
```

tells Spring:

"This controller handles URLs starting with /email."

```
@PostMapping("/analyze")
```

tells Spring:

"When a POST request comes to /analyze, execute this method."

Spring combines them:

```
/email  
+  
/analyze
```

becomes:

```
/email/analyze
```

and automatically calls your method when that URL is requested.

Next section:

◆ **analyze() Method Line-by-Line Explanation**

This starts with:

```
public ResponseEntity<EmailResponseDTO> analyze(  
    @RequestBody EmailRequestDTO request)
```

and explains every single word (**public**, **ResponseEntity**, generics, method name, parameter, **@RequestBody**, etc.).